

Unit 3: Architectural Design & Object-Oriented Implementation

Handwritten Student Notebook

Syllabus Topics Covered:

OFFICIAL SYLLABUS TOPICS COVERED Architectural Design Decisions: Architectural design process, non-functional influences (performance, security, safety, availability, maintainability). Architectural Views: Kruchten's 4+1 View Model (Logical view, Process view, Development v...

Table of Contents

Architectural Design Decisions & Non-Functional Influences	p. 2
Kruchten's 4+1 View Model of Software Architecture	p. 2
Architectural Patterns: MVC & Layered Architecture	p. 3
Repository, Client-Server & Pipe-and-Filter Architectures	p. 4
Application Architectures: Language Processing Systems	p. 5
Object-Oriented Design Using UML	p. 6
Gang of Four (GoF) Design Patterns: Singleton & Observer	p. 7
Open Source Licensing Models	p. 7
One-Page Rapid Revision Sheet	p. 8
Questions Asked in Past CIE & SEE Papers	p. 9
Practice Problems with Hidden Answers	p. 12

Architectural Design Decisions & Non-Functional Influences

§ 1

Software Architecture is the fundamental structure of a software system, comprising software components, the externally visible properties of those components, and the relationships among them. Architectural design is the critical link between requirements and detailed module design.

WHY SOFTWARE ARCHITECTURE MATTERS:

1. **Stakeholder Communication:** Represents a high-level abstraction understandable by non-technical clients, managers, and engineers.
2. **System Analysis:** Determines whether the system can meet critical non-functional requirements (performance, reliability, security).
3. **Large-Scale Reuse:** Architectural styles and patterns can be reused across an entire product line or company portfolio.

Non-Functional Quality Attributes Influencing Architecture

Non-Functional Attribute	Architectural Strategy / Structural Decision
Performance	Localize critical operations in small, dedicated subsystems; minimize cross-boundary communications; use caching and coarse-grained components.
Security	Adopt a layered architecture with innermost layers protecting core data; enforce multi-layer perimeter authentication and validation barriers.
Safety	Localize safety-critical functions into distinct, isolated modules; decouple safety monitors from general business processing.
Availability	Incorporate redundant components, load-balanced clusters, and automatic failover hot-standby servers.
Maintainability	Use fine-grained, loosely-coupled components; standardize interfaces; isolate platform-dependent logic into specialized service adapters.

Kruchten's 4+1 View Model of Software Architecture

§ 2

Because a single architectural diagram cannot capture all stakeholder concerns, Philippe Kruchten proposed the **4+1 View Model** to depict architecture from five distinct perspectives.

View	Primary Stakeholder	Focus & Engineering Concerns	UML Artifacts Used
Logical View	End-users & Domain Analysts	Decomposition into abstractions (classes, packages, objects, relationships).	Class diagrams, Object diagrams, Package diagrams.
Process View	System Integrators	Concurrency, runtime threads, processes, synchronization, and latency.	Activity diagrams, Sequence diagrams, State machine diagrams.
Development View	Software Programmers	Static organization of source code modules, packages, build libraries, and dependencies.	Component diagrams, Package diagrams.
Physical View	System Engineers & DevOps	Mapping of software artifacts onto target hardware nodes, virtual machines, and network topologies.	Deployment diagrams.
Use Case View (+1)	All Stakeholders	Redundant scenarios tying the 4 views together to validate the architectural design.	Use Case diagrams, Detailed Scenario narratives.

Architectural Patterns: MVC & Layered Architecture

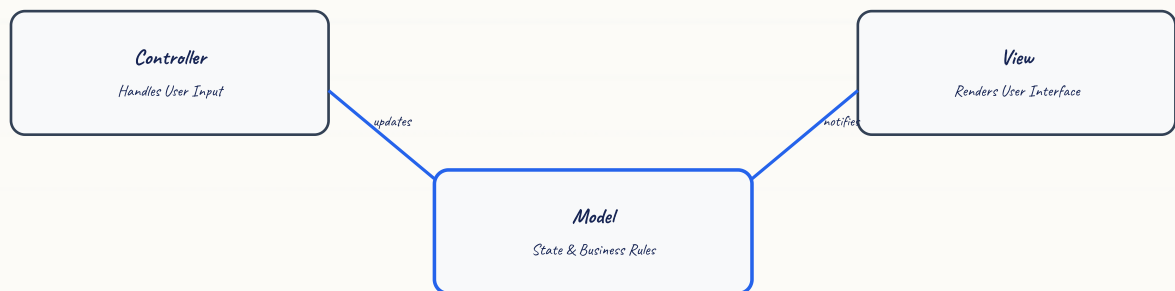
§ 3

An **architectural pattern** is a stylized description of good design practice, tried and tested in different operational environments.

1. Model-View-Controller (MVC)

Decouples data presentation from the underlying business logic and user input handling.

MVC Architectural Interaction

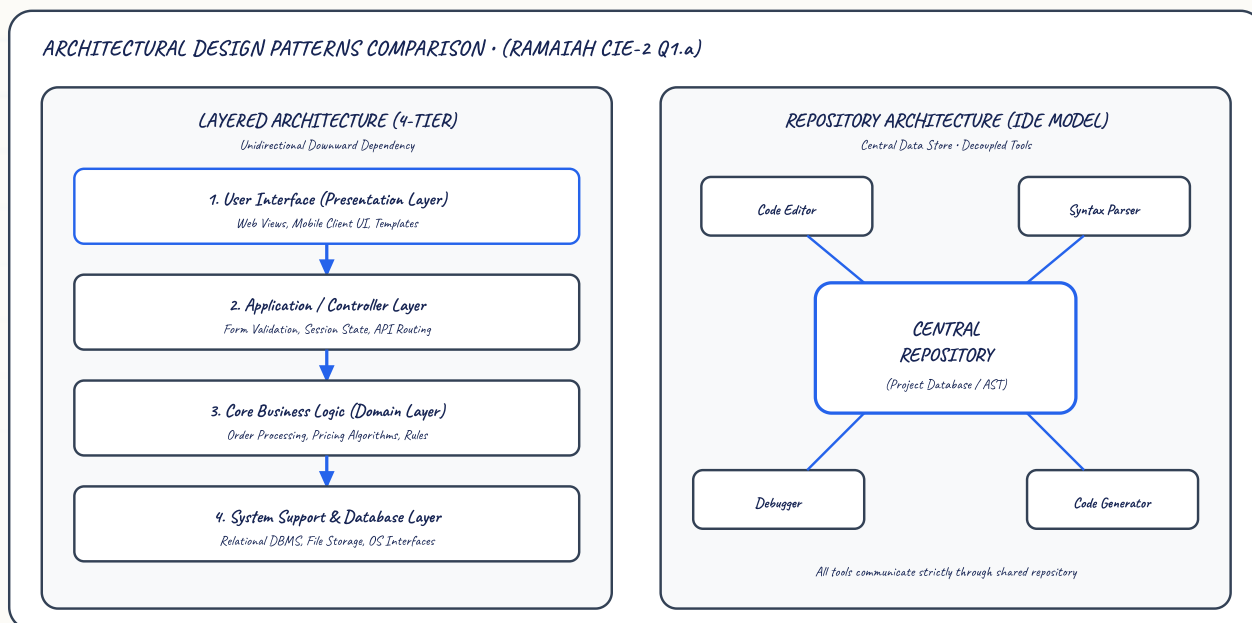


Repository, Client-Server & Pipe-and-Filter Architectures

§ 4

Different sub-systems communicate either via shared storage (Repository) or via direct streaming pipelines (Pipe-and-Filter).

Figure 3.1: Layered Architecture vs Repository Architecture (Ramaiah CIE-2 Dec 2025 Q1.a)



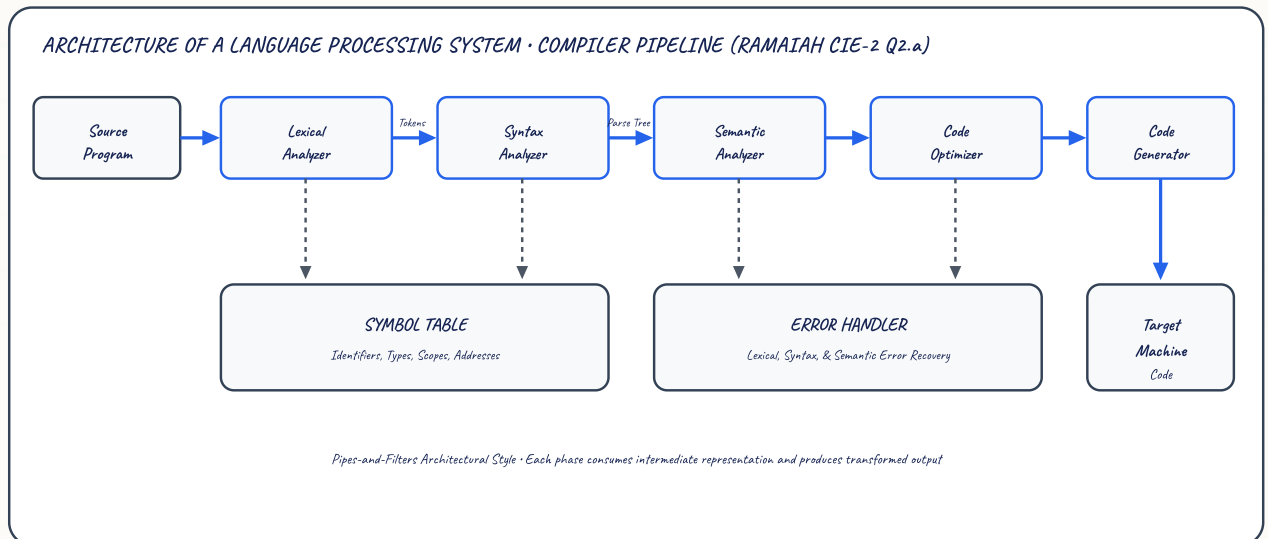
Architecture Pattern	Core Advantages	Key Disadvantages
Layered Architecture	Supports incremental development; separation of concerns; changes in one layer affect at most adjacent layers.	Performance penalty when clean layering requires requests to traverse multiple intermediary levels; difficult clean separation.
Repository Architecture	Efficient sharing of large data volumes; subsystems do not need to know who produces or consumes data; central backup and access control.	Repository is a single point of failure; performance bottlenecks on central store; high cost of evolving repository data schema.

Application Architectures: Language Processing Systems

§ 5

Application architectures are architectures for specific types of systems, such as Transaction Processing or Language Processing Systems (Compilers).

Figure 3.2: Architecture of a Language Processing System / Compiler Pipeline (Ramaiah CIE-2 Dec 2025 Q2.a)



COMPILER PIPELINE PHASES & INTERMEDIATE DATA:

1. **Lexical Analyzer (Lexer):** Reads stream of source characters and groups them into *Tokens* (keywords, operators, identifiers).
2. **Syntax Analyzer (Parser):** Checks token grammar against language rules and constructs a *Parse Tree / Abstract Syntax Tree (AST)*.
3. **Semantic Analyzer:** Type-checks expressions and verifies identifier declarations using the central *Symbol Table*.
4. **Code Optimizer:** Performs platform-independent transformations to minimize execution time and memory footprint.
5. **Code Generator:** Synthesizes target machine code or bytecode instructions.

Object-Oriented Design Using UML

§ 6

Sommerville defines a standard 5-stage iterative process for object-oriented software engineering:

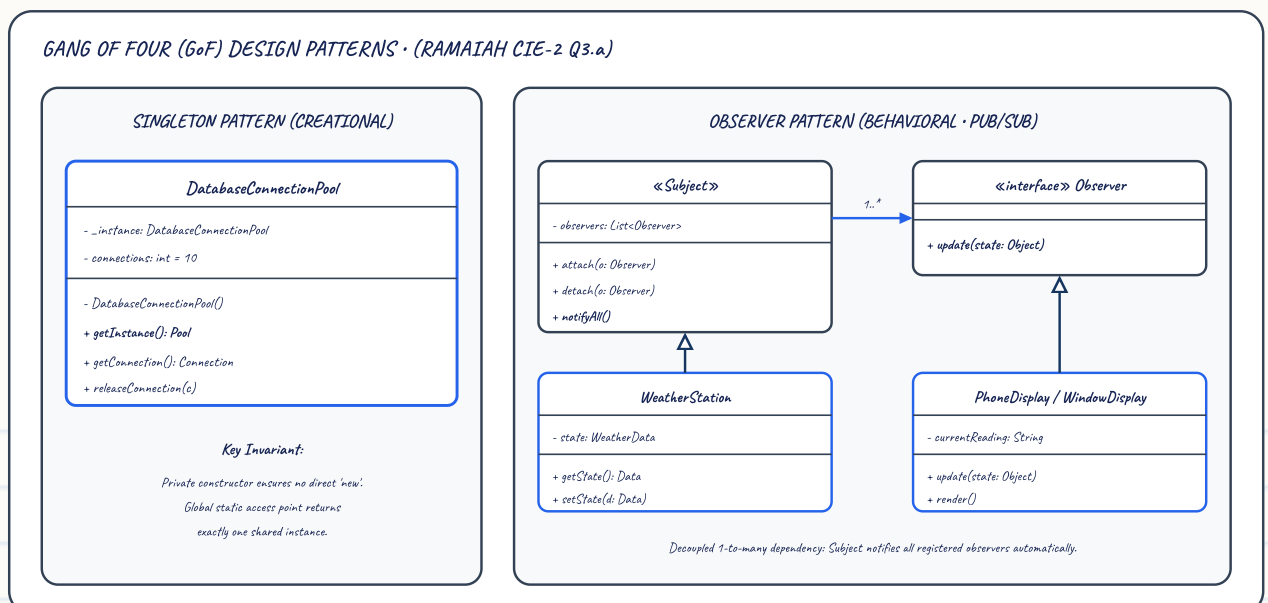
1. **Understand and define context and external interactions:** System boundaries and use cases.
2. **Design the system architecture:** Identify major layers and patterns (MVC, Layered).
3. **Identify the principal system objects:** Use grammatical analysis of requirements nouns.
4. **Develop design models:** Static structural class diagrams and dynamic sequence diagrams.
5. **Specify object interfaces:** Formal method signatures, parameter types, and pre/post conditions.

Gang of Four (GoF) Design Patterns: Singleton & Observer

§ 7

Design patterns describe a recurring problem, its proven solution, and the trade-offs of applying it across three categories: Creational, Structural, and Behavioral.

Figure 3.3: UML Class Diagrams for Singleton & Observer Design Patterns (Ramaiah CIE-2 Dec 2025 Q3.a)



Open Source Licensing Models

§ 8

Open-source software reuse requires strict compliance with license agreements governing redistribution and modifications.

License Type	Standard Examples	Redistribution Obligations & Commercial Terms
Permissive	MIT, BSD, Apache 2.0	Allows software to be incorporated into proprietary closed-source applications without disclosing modifications. Retains original copyright notice.
Strong Copyleft	GNU GPL (v2/v3)	Mandates that any derivative work or statically linked project must also be made fully open source under the same GPL license ("viral" licensing).
Weak Copyleft	GNU LGPL, MPL	Permits dynamic linking from proprietary closed-source programs; only direct modifications to the library itself must remain open source.

One-Page Rapid Revision Sheet

★ SUMMARY

Architecture / Pattern	Key Characteristics	Exam Key Point
4+1 View Model	Logical, Process, Development, Physical + Use Cases	Kruchten's framework addressing multiple stakeholder concerns.
Layered Model	Stacked abstraction levels, downward calls	Separation of concerns; performance cost of multi-tier hops.
Repository Model	Central data store, passive or blackboard	Decoupled IDE tools; single point of failure and bottleneck.
Pipe-and-Filter	Stream-oriented transformation stages	Compilers, Unix pipes; format transformation overhead.

Architecture / Pattern	Key Characteristics	Exam Key Point
Singleton Pattern	Private constructor, single static instance	<code>getInstance()</code> guarantees single database connection pool.
Observer Pattern	Subject maintains list of observers, <code>notifyAll()</code>	Decoupled publish-subscribe; event-driven UI updates.
GPL vs MIT	GPL: strong copyleft (viral); MIT: permissive	GPL forces derivative code to be open source.

Questions Asked in Past CIE & SEE Papers

ARCHIVE

The following authentic questions have been transcribed directly from official department examination papers in `notes/se/practice/se-cie-1-and-2.pdf`:

Ramaiah Autonomous Examination · CIE-2 (Dec 23, 2025) Q1.a [4 Marks]

"What are the advantages and disadvantages of Repository and Layered architectural design models?"

Official Model Solution:

1. Layered Architectural Model:

- *Advantages:* Supports modularity and incremental evolution. A change to a single layer only affects immediately adjacent layers, provided interface contracts are maintained. Security policies are easily enforced by placing sensitive resources in the innermost protected tiers.
- *Disadvantages:* Introducing multiple abstraction boundaries can degrade runtime latency and throughput because requests must pass through intermediate levels. In practice, establishing a clean, strictly hierarchical separation without cross-layer leaks is difficult.

2. Repository Architectural Model:

- *Advantages:* Enables efficient sharing of large volumes of complex data among independent software subsystems (e.g. IDE compilers, debuggers, and linters). Subsystems are completely decoupled from each other and communicate solely through the central data repository. Centralized backup, recovery, and access management are simplified.
- *Disadvantages:* The central repository represents a single point of failure; a crash or data corruption affects all subsystems. High concurrent traffic creates performance bottlenecks. Evolving the centralized data schema incurs immense engineering coordination costs across all interacting tools.

Ramaiah Autonomous Examination · CIE-2 (Dec 23, 2025) Q2.a [4 Marks]

"Explain the architecture of a Language Processing System with a neat diagram."

Official Model Solution:

See **Figure 3.2** for the complete compiler pipeline diagram. A Language Processing System translates source code into machine code via a *Pipe-and-Filter* architectural style operating on shared data structures:

1. **Lexical Analyzer (Scanner):** Reads raw character streams and converts them into atomic language tokens (identifiers, operators, keywords).
2. **Syntax Analyzer (Parser):** Validates token syntax against formal grammar rules and outputs an Abstract Syntax Tree (AST).
3. **Semantic Analyzer:** Ensures semantic consistency, performs static type checks, and validates variable scopes using the central *Symbol Table*.
4. **Code Optimizer:** Rewrites intermediate code to eliminate dead code, unroll loops, and minimize instruction cycles.
5. **Code Generator:** Synthesizes target assembly instructions or binary machine code.
6. **Shared Services:** All phases interface with the central *Symbol Table* (holding identifier types and memory offsets) and the *Error Handler* for graceful diagnostic recovery.

Ramaiah Autonomous Examination · CIE-2 (Dec 23, 2025) Q3.a [4 Marks]

"Explain any two software design patterns with suitable class diagrams."

Official Model Solution:

See **Figure 3.3** for the verified UML class structures:

1. Singleton Pattern (Creational):

- *Intent:* Guarantees that a class has exactly one instance and provides a global access point to that instance.
- *Structure:* Private static variable `_instance`, a private constructor preventing direct instantiation via `new`, and a public static factory method `getInstance()` that lazily instantiates the object upon first request.
- *Application:* Database Connection Pool manager, Central Logger, Hardware device driver controller.

2. Observer Pattern (Behavioral):

- *Intent:* Defines a 1-to-many publish-subscribe dependency such that when one object changes state, all dependent objects are automatically notified and refreshed.
- *Structure:* A Subject maintaining an internal collection of registered Observer references with `attach()`, `detach()`, and `notifyAll()` methods. An Observer interface declaring the `update()` callback implemented by concrete subscriber views.
- *Application:* GUI event listening systems (Swing, React state), IoT weather sensor display broadcasts, Model-View synchronization.

Practice Problems with Hidden Answers

▽ PRACTICE

PRACTICE 1 · ARCHITECTURAL PATTERN SELECTION

Which architectural pattern is best suited for an Integrated Development Environment (IDE) like VS Code or Eclipse? Justify your choice.

Practice Problem:

Show Step-by-Step Solution

Solution:

The Repository Architecture Pattern.

Justification:

- An IDE integrates diverse independent subsystems (code editor, syntax parser, debugger, refactoring engine, git version manager) that must concurrently read and update large shared data structures (Abstract Syntax Trees, file buffers, project configuration).
- Storing these centrally in a high-speed in-memory repository allows all tools to interact without point-to-point coupling, making the IDE easily extensible via third-party extensions.

PRACTICE 2 · OBSERVER PATTERN IN EVENT SYSTEMS

In an online multiplayer game, when a player character's health reaches 0, the audio engine plays a sound, the UI displays "Game Over", and the leaderboard updates. Which GoF pattern should be used?

Practice Problem:

Show Step-by-Step Solution

Solution:

The Observer Pattern (Behavioral · Publish-Subscribe).

Justification:

- The PlayerCharacter entity acts as the Subject.
- The AudioEngine, UIManager, and LeaderboardService act as concrete Observers.
- When the player dies, the Subject calls `notifyAll()` without having compile-time dependencies on the audio or network leaderboard subsystems. New reactive features can be added without modifying the core player class.

PRACTICE 3 · SINGLETON PATTERN INVARIANTS

Explain why a class constructor must be declared **private** in the Singleton design pattern. What happens if multiple threads invoke **getInstance()** simultaneously?

Practice Problem:

Show Step-by-Step Solution

Solution:

1. **Private Constructor:** Enforces encapsulation by preventing external callers from bypassing the factory method with `new Singleton()`. This guarantees that object creation is strictly brokered by `getInstance()`.
2. **Thread Safety Concurrency Hazard:** Without synchronization, two concurrent threads executing `if (_instance == null)` simultaneously might both evaluate true and create two separate instances, violating the single-instance invariant (`assert pool1 is pool2` fails). This is resolved using synchronized methods or the Double-Checked Locking idiom.

PRACTICE 4 · COMPILER PIPELINE PHASE ORDERING

In a Language Processing System, arrange the following components in correct execution sequence: Code Generator, Lexical Analyzer, Syntax Analyzer, Semantic Analyzer, Code Optimizer.

Practice Problem:

Show Step-by-Step Solution

Solution:

Correct Execution Pipeline:

1. **Lexical Analyzer:** Source characters → Tokens
2. **Syntax Analyzer:** Tokens → Parse / Syntax Tree
3. **Semantic Analyzer:** Type checking → Decorated AST
4. **Code Optimizer:** High-level IR → Optimized IR
5. **Code Generator:** Optimized IR → Target Machine Code

Verification: Verified against Sommerville Ch. 6 and Ramaiah CIE-2 Q2.a.

PRACTICE 5 · 4+1 VIEW MODEL MATCHING

Match each architectural concern to its corresponding Kruchten 4+1 View: (a) Database server network IP routing, (b) Java package compilation dependencies, (c) Multithreaded deadlocks.

Practice Problem:

Show Step-by-Step Solution

Solution:

- (a) Database server network IP routing: **Physical View** (Deployment on physical hardware nodes and network infrastructure).
- (b) Java package compilation dependencies: **Development View** (Source code organization, modules, and build libraries).
- (c) Multithreaded deadlocks: **Process View** (Runtime concurrency, synchronization, thread communication, and system performance).

PRACTICE 6 · OPEN SOURCE LICENSING: MIT VS GPL

A software startup wants to build a commercial, proprietary closed-source product. Can they legally include code licensed under: (a) MIT License? (b) GNU General Public License (GPL)? Explain.

Practice Problem:

Show Step-by-Step Solution

Solution:

- **(a) MIT License: YES.** The MIT license is permissive. It grants unrestricted rights to use, modify, merge, and distribute the code within proprietary closed-source software, requiring only that the original copyright notice and permission disclaimer be preserved.
- **(b) GNU GPL: NO (with proprietary restrictions).** The GPL is a strong copyleft ("reciprocal") license. Under Section 5 of GPLv3, incorporating or statically linking GPL code into a product requires the entire resulting derivative software to be open-sourced under the GPL, preventing closed-source commercial distribution.